MIDI Generation Architecture

Technical Deep Dive (`src/music/`)

This document outlines the complete technical pipeline for transforming real-time emotional vectors (Valence and Arousal) into cohesive MIDI sequences using both rule-based heuristics and Markov-chain statistical models.

Directory Context & Core Files

The `src/music/` directory is responsible for all musical mapping and rendering.

- `emotion_tracker.py` - Moving average filters and spike detection.
- `midi_generator.py` - Offline generation pipeline (exports `.mid` files).
- `realtime_generator.py` - Queued real-time MIDI scheduling via FluidSynth.
- `train_markov_midi.py` - Extractor script that builds transition profiles from the VGMIDI dataset.
- `markov_engine.py` - Read-only statistical engine querying dataset transition weights.

---

1. Emotion Processing (`emotion_tracker.py`)

Because raw EEG outputs are highly noisy and jittery, mapping them strictly 1:1 to MIDI parameters creates auditory chaos. The `EmotionTracker` maintains a sliding window `collections.deque` (default size = 10 samples) to separate the signal:

- `macro_v` and `macro_a`: The mean of the window. Represents the foundational emotional state. This dictates heavy structural changes (key signatures, chord progressions).
- `micro_v` and `micro_a`: The immediate delta (current sample minus the macro average). Represents transient spikes. This triggers instantaneous expression (velocity surges, octave jumps, dissonant accidentals) without derailing the harmonic foundation.

---

2. Harmonic & Accompaniment Rules

Section 3 of the generative engines maps the `Macro_Mood` to specific musical behavior.

Emotion Classifications:

- **Happy** (`v >= 0, a >= 0`)
- **Sad** (`v < 0, a < 0`)
- **Fear** (`v < 0, a >= 0`)
- **Neutral** (`abs(v) < 0.2, abs(a) < 0.2`)

Rule-Based Chord Progressions: To break away from pure random generation, each quadrant dictates an explicit, stylistic chord sequence:

- `Happy`: [I, V, vi, IV]
- `Sad`: [i, VII, VI, v]
- `Fear`: [i, i, bII, i] (Static pedaling with Neapolitan tension)
- `Neutral`: [II, I, IV, V]

Accompaniment Styles:

- `sustained_block` (Happy): Heavy, grounded triads.
- `open_wide` (Sad): Root + descending 5th + 3rd in the next octave (Tragic spacing).
- `drone_pedal` (Fear): Static root sustained across measures. Micro-arousal spikes trigger a heavy MIDI velocity swell (rather than a melodic trill).
- `quartal_float` (Neutral): Chords stacked in perfect fourths (Root, 4th, b7) to strip away major/minor categorization.

---

3. The Markov Melody Engine

To prevent the right-hand melody from sounding like an unmusical random walk, the generator uses a first-order Markov Chain trained on the **VGMIDI Dataset**.

Extraction Pipeline (`train_markov_midi.py`)

The extraction script processes 200+ MIDI files alongside continuous 32-sample Valence-Arousal annotations (averaged across 30 annotators per piece).

1. **Windowing:** The script divides the MIDI file into 32 physical time windows matching the annotation timestamps.
2. **Labeling:** Each window is tagged with an emotion quadrant based on the specific V-A values at that moment in time.
3. **Transition Counting:** For every consecutive note pair in that window, it counts `current_pitch_interval` â†' `next_pitch_interval`.
4. **Export:** Normalises the counts into probabilities and exports 4 JSON matrices (e.g. `transitions_happy.json`).

Real-Time Querying (`markov_engine.py`)

At startup, `MarkovEngine` loads the JSONs into memory. During the Melody section of the `realtime_generator`:

1. The generator observes the current `macro` quadrant.
2. It tracks `prev_melody_interval` (the distance between the last two notes played).
3. It queries the Markov Engine: `markov_engine.query_next_interval(emotion_cat, prev_interval)`
4. The engine uses `random.choices` fueled entirely by the VGMIDI transition weights to dictate the next step size.

Melodic Bounds & Filters

While the VGMIDI matrix controls the *interval jumps*, absolute harmonic adherence is enforced by strict Python bounds:

1. `micro_v` chromatic substitutions are laid *over* the Markov suggestion.
2. If the quadrant is `Neutral`, the active melodic pool is dynamically locked to a strictly

Pentatonic scale. The Markov interval shift simply advances the index along the filtered pentatonic pool.

---

4. Live Queuing Architecture (`realtime_generator.py`)

The real-time synth must bridge the asynchronous gap between EEG inputs (e.g., 2Hz) and precise musical timing.

It accomplishes this via a `QThread` event loop:

- **`update_queue`:** Captures V-A states from the frontend.
- **`_generate_and_schedule_1s_chunk`:** Unpacks the V-A state to calculate the harmonic rhythm, queries the `MarkovEngine`, applies the accompaniment rules, and generates a list of absolute timestamps (e.g., `current_time + 0.25s: NOTE ON, current_time + 0.49s: NOTE OFF`).
- These absolute events are appended to `note_queue` and sorted.
- The main `run()` loop uses spinning `time.sleep(0.005)` queries. When `time.time() >= event.timestamp`, the Python `fluidsynth` binding fires the MIDI command to the OS audio driver with near-zero latency.